

Rapport – Introduction à Pharo

1. Les collections en Pharo

- Qu'est-ce qu'une collection et à quoi sert-elle ?

Une collection est un objet dont le rôle est de regrouper et de gérer d'autres objets. C'est une structure de données fondamentale qui permet de stocker, accéder, trier et manipuler efficacement des groupes d'éléments.

- Quels types de collections la bibliothèque standard de Pharo fournit-elle ?

Pharo propose une hiérarchie riche et variée de collections. Les plus courantes sont :

Array , OrderedCollection ,Set , Dictionary

Comment itérer sur les collections et quelles sont les différences ?

En Pharo, l'itération se fait en envoyant des **messages** à une collection, en lui passant un **bloc** de code.

Quelques itérateurs essentiels :

- `do:` → exécute un bloc pour chaque élément.
- `collect:` → transforme chaque élément en un nouveau et retourne une nouvelle collection.
- `select:` → retourne une collection avec seulement les éléments pour lesquels le bloc est vrai.

2. Les conditionnels en Pharo

- Comment écrire des conditionnels en Pharo ?

En Pharo, il n'existe pas de mot-clé `if` comme dans d'autres langages.

À la place, on envoie des messages aux objets booléens `true` et `false` :

- `ifTrue:`
- `ifFalse:`

(temperature > 20)

`ifTrue: [ Transcript show: 'Il fait chaud' ]`

`ifFalse: [ Transcript show: 'Il fait frais' ].`

- Avantages et inconvénients

Avantages :

- Cohérence objet : tout est message envoyé à un objet.
- Extensibilité : possibilité de définir des objets booléens personnalisés.

Inconvénients :

- Courbe d'apprentissage : approche déroutante pour un développeur venant de Java ou Python.
- Lisibilité : certains trouvent `if (condition) {...}` plus clair.

3. Créer des classes et des méthodes en Pharo

Les classes se créent dans des **packages**, et les méthodes se définissent côté **instance** ou côté **classe**.

4. Style de code en Pharo

Pharo encourage un style clair et lisible :

- Méthodes **courtes** et centrées sur une seule responsabilité.
- Les **noms de méthodes** doivent être explicites.
- Pas de **variables inutilisées**.
- Indentation correcte et homogène

5. Extras

Cascades

Une **cascade (.)** permet d'envoyer plusieurs messages au même objet sans le répéter.

Block closures

Un **bloc** est une portion de code entre crochets `[ ]`, semblable à une fonction anonyme.

Les blocs sont essentiels pour les **boucles**, **conditionnels** et **itérateurs**.

- J'ai trouvé la majorité des informations dans la **documentation en ligne**.
- J'ai également consulté des **vidéos en ligne**.